

Divide and Conquer

TDA - Algo3

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2do Cuatrimestre de 2026

Plan

- 1 Técnica algorítmica: Divide & Conquer.
 - Qué es una técnica algorítmica?
 - El patrón de D&C en el caso de merge sort.
 - Cálculo de la complejidad vía sustitución.
- 2 Algoritmo de Karatsuba
 - El Teorema Maestro para resolver recurrencias
- 3 Pausa
- 4 El algoritmo de Strassen.
- 5 Timba financiera.
- 6 Búsqueda binaria.
- 7 Conclusión

Basado en el Cormen! Léanlo (o lean, en general).

Técnicas algorítmicas

A medida que resolvemos problemas, nos vamos dando cuenta de que hay **patrones** o **enfoques** que se repiten una y otra vez. A estos patrones los reificamos como **técnicas algorítmicas**.

Definición (informal)

Una **técnica algorítmica** es un enfoque general para implementar un algoritmo.

No son categorías perfectamente definidas. En particular, hay algoritmos que no se pueden pensar como asociados a una técnica específica, o bien pueden ser una mezcla de varias.

Técnicas algorítmicas

En general, vemos técnicas porque:

- Son patrones de solución que han funcionado en otros problemas, por lo que tiene sentido intentar aplicarlos a nuevos.
- En general vienen asociados a conjuntos de herramientas y prácticas para probar correctitud y complejidad. **Ejemplo:** los algoritmos recursivos, cuya correctitud se prueba por inducción y cuya complejidad sale resolviendo alguna relación de recurrencia.
- Facilitan la comunicación y la comprensión de los algoritmos. **Ejemplo:** “Hago recursión en n , fijate que $f(n)$ depende de $f(n - 4)$ y de $f(n - 1)$, así que es inmediato cómo calcularlo”.

Técnicas algorítmicas

En general, vemos técnicas porque:

- Son patrones de solución que han funcionado en otros problemas, por lo que tiene sentido intentar aplicarlos a nuevos.
- En general vienen asociados a conjuntos de herramientas y prácticas para probar correctitud y complejidad. **Ejemplo:** los algoritmos recursivos, cuya correctitud se prueba por inducción y cuya complejidad sale resolviendo alguna relación de recurrencia.
- Facilitan la comunicación y la comprensión de los algoritmos. **Ejemplo:** “Hago recursión en n , fijate que $f(n)$ depende de $f(n - 4)$ y de $f(n - 1)$, así que es inmediato cómo calcularlo”.

En la clase de hoy vamos a ver Divide&Conquer (D&C): daremos la idea general del patrón, y distintos casos de uso. Aparte, vamos a ver técnicas y teoremas que permiten calcular complejidades de forma sencilla.

Divide&Conquer

Esquemáticamente, un algoritmo de D&C tiene las siguientes partes.

1. Dividir

Separar la instancia en subproblemas más pequeños del mismo tipo.

2. Conquistar

Resolverlos recursivamente; los casos pequeños se resuelven directamente.

3. Combinar

Reagrupar las soluciones parciales para obtener la solución original.

En el fondo, es solo recursión, pero vamos a ver que es muy común que los subproblemas tengan todos **el mismo tamaño**, y que la complejidad general suele depender únicamente de (1) la cantidad de subproblemas, (2) su tamaño, y (3) el costo de combinar las soluciones.

Divide&Conquer

Los algoritmos de D&C suelen tener esta pinta.

DIVIDIR-Y-CONQUISTAR(X)

```
if  $X$  es un caso base  
    return RESOLVER( $X$ )  
 $(X_1, \dots, X_a) \leftarrow \text{DIVIDIR}(X)$   
para  $i = 1, \dots, a$   
     $Y_i \leftarrow \text{DIVIDIR-Y-CONQUISTAR}(X_i)$   
return COMBINAR( $Y_1, \dots, Y_a$ )
```

Típicamente, se demuestra

- **Correctitud**, por inducción, usando la recursión.
- **Complejidad**: calculando el tamaño de cada $X_1 \dots X_a$, y el costo de COMBINAR (en general, el caso base suele ser $O(1)$, aunque podría no serlo).

Ejemplo de D&C: Merge sort

Problema: Sorting

Dado un arreglo de enteros A , nos piden ordenarlo de menor a mayor.

Ya conocen varias formas de hacer esto. Una de las posibles es la siguiente:

- 1 **Dividir:** partir el arreglo A de longitud n en dos subarreglos A_1 y A_2 de longitud $n/2$.
- 2 **Conquistar:** ordenar A_1 y A_2 .
- 3 **Combinar:** combinar los dos arreglos ordenados.

Ejemplo de D&C: Merge sort

Siendo más precisos, vamos a diseñar un algoritmo $\text{MERGESORT}(A, l, r)$ que recibe un arreglo A y dos índices l, r con $l \leq r$ que ordena el subarreglo $A[l : r]$. Con esta notación, siguiendo la idea de la slide anterior el algoritmo es

```
MERGESORT( $A, l, r$ )
```

```
if  $l = r$ 
```

```
    return
```

```
 $q \leftarrow \lfloor (p + r) / 2 \rfloor$ 
```

```
MERGESORT( $A, l, q$ )
```

```
MERGESORT( $A, l + 1, r$ )
```

```
MERGE( $A, l, q, r$ )
```

```
return
```

Cómo se hace el MERGE al en la última línea?

Ejemplo de D&C: Merge sort

Se puede implementar en tiempo lineal: copiamos $A[l \dots q]$ en L y $A[q + 1 \dots r]$ en R . Luego, recorremos L y R a la vez y vamos eligiendo siempre el elemento más chico.

MERGE (idea)

$k = l; i, j = 0;$

while $(i < |L| \wedge j < |R|)$

if $L[i] \leq R[j]$

$A[k] \leftarrow L[i]; i \leftarrow i + 1$

else

$A[k] \leftarrow R[j]; j \leftarrow j + 1$

$k \leftarrow k + 1$

Finalmente, copiar lo que queda al final del subarreglo de A .

Esto cuesta tiempo lineal.

Ejemplo de D&C: Merge sort

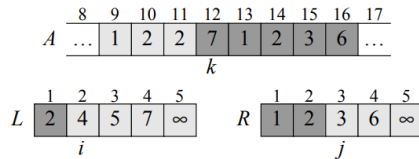
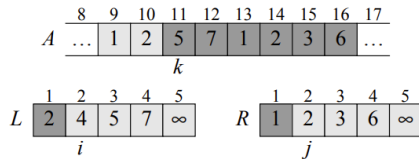
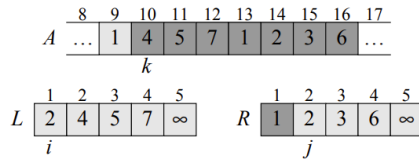
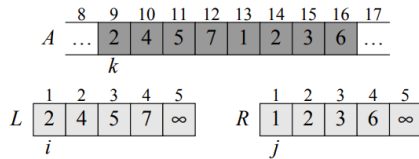


Figura: Ejemplo de MERGE

Ejemplo de D&C: Merge sort

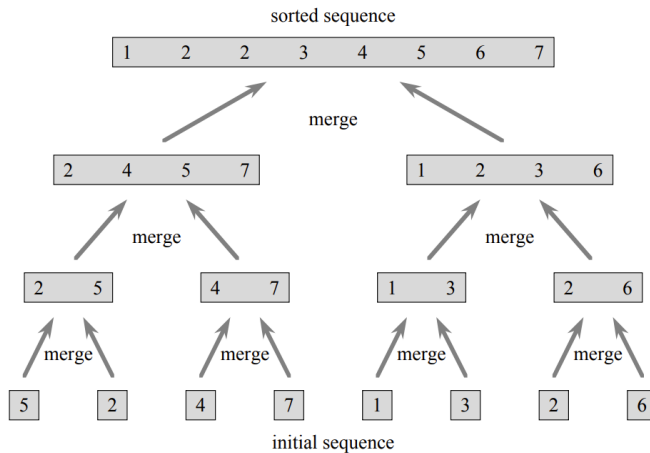


Figura: Ejemplo de MERGESORT

Ejemplo de D&C: Merge sort

Este algoritmo sigue el **patrón** de D&C. Veamos la cantidad de operaciones que se hace en cada componente del algoritmo.

Dividir

Calcular el punto medio
cuesta $\Theta(1)$.

Conquistar

Hay dos llamadas sobre
instancias de tamaño $n/2$.

Combinar

MERGE toma $\Theta(n)$.

Luego, la complejidad de MERGESORT satisface la siguiente recurrencia²

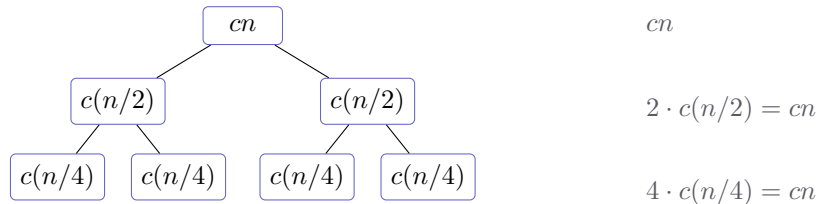
$$T(n) = 2T(n/2) + \Theta(n), \quad T(1) = \Theta(1).$$

Cómo encontramos una cota fina a esta complejidad?

²En realidad deberíamos usar $\lfloor n/2 \rfloor$ y $\lceil n/2 \rceil$, pero en este caso (y en la mayoría de los de la materia) es irrelevante en términos de complejidad de peor caso.

Ejemplo de D&C: Merge sort

Podemos pensar en el árbol de recursión, e intentar llevar la cuenta de cuánto se trabaja en cada llamado. Si frente a un arreglo de n números se hacen cn operaciones (ignorando la recursión), entonces tenemos los siguientes costos



En cada “piso” se hacen cn operaciones, y hay $\log n$ pisos. Luego, la complejidad parecería ser $O(n \log n)$. Vamos a probarlo rigurosamente

Método de sustitución

Supongamos que tenemos alguna recurrencia, como por ejemplo

$$T(n) = 2T(n/2) + \Theta(n).$$

Si ya tenemos una cota $T(n) = O(f(n))$ candidata, podemos intentar probarla directamente por inducción.

El método de sustitución

- 1 Conjeturamos una cota. Es decir, una función $f(n)$ y una constante C .
- 2 La usamos como hipótesis inductiva: para acotar $T(n)$ la reescribimos recursivamente y usamos la hipótesis.

Se llama *sustitución* porque en el paso inductivo vamos a acotar / sustituir $T(n/2)$ por $Cf(n/2)$.

El método de sustitución

Sea $d = T(2)$ la cantidad de operaciones en el caso base de merge sort, y recordemos que existe una c tal que

$$T(n) \leq 2T(n/2) + cn.$$

Luego, tomemos $f(n) = n \log n$ y $C = \max \{c, d\}$ y probemos que $T(n) \leq Cn \log n$, lo cual implicaría que $T(n) = O(n \log n)$. Lo hacemos por inducción (con caso base $n = 2$, ya que $\log 1 = 0$).

El caso base $n = 2$ vale por elección de d . Para el caso inductivo, tenemos que

$$\begin{aligned} T(n) &\leq 2T(n/2) + cn \\ &\leq 2 \left(C \frac{n}{2} \log_2 \frac{n}{2} \right) + cn \\ &\leq Cn \log_2 \frac{n}{2} + Cn \\ &\leq Cn \left(\log_2 \frac{n}{2} + 1 \right) = Cn \log_2 n, \end{aligned}$$

Por lo que probamos el caso inductivo. Luego, la complejidad de MERGESORT es ▶

Algoritmo de Karatsuba

Problema: producto de números

Sean x e y números naturales de n dígitos. Queremos calcular su producto xy .

Ya conocemos un algoritmo para hacer esto: el de la escuela. Qué complejidad tiene?³

³De paso: qué complejidad tiene sumar, en comparación?

Algoritmo de Karatsuba

Problema: producto de números

Sean x e y números naturales de n dígitos. Queremos calcular su producto xy .

Ya conocemos un algoritmo para hacer esto: el de la escuela. Qué complejidad tiene?³

Se puede implementar en tiempo $O(n^2)$. Ahora, intentemos hacer algo mejor usando D&C.

³De paso: qué complejidad tiene sumar, en comparación?

Algoritmo de Karatsuba

Sea $m \approx n/2$. Podemos escribir los dos números como

$$x = x_1 2^m + x_0, \quad y = y_1 2^m + y_0.$$

Luego, usando distributiva tenemos que

$$xy = x_1 y_1 2^{2m} + (x_1 y_0 + x_0 y_1) 2^m + x_0 y_0.$$

Esto nos sugiere un algoritmo de D&C: calculamos los 4 productos de números de $n/2$ dígitos, y luego mergear las soluciones con la suma y los *shifteos*.

Algoritmo de Karatsuba

Sea $m \approx n/2$. Podemos escribir los dos números como

$$x = x_1 2^m + x_0, \quad y = y_1 2^m + y_0.$$

Luego, usando distributiva tenemos que

$$xy = x_1 y_1 2^{2m} + (x_1 y_0 + x_0 y_1) 2^m + x_0 y_0.$$

Esto nos sugiere un algoritmo de D&C: calculamos los 4 productos de números de $n/2$ dígitos, y luego mergear las soluciones con la suma y los *shifteos*.

Cuatro subproductos

$x_1 y_1, x_1 y_0, x_0 y_1$ y $x_0 y_0$.

Combinar

Es $\Theta(n)$.

La cantidad de pasos del algoritmo satisface la recursión $T(n) = 4T(n/2) + \Theta(n)$. Si la resolvemos, solo la acotamos por $O(n^2)$...

Algoritmo de Karatsuba

Alternativa: Calculamos recursivamente sólo

$$z_0 = x_0y_0, \quad z_2 = x_1y_1, \quad z_s = (x_0 + x_1)(y_0 + y_1).$$

Como

$$z_s = z_0 + z_2 + x_0y_1 + x_1y_0,$$

obtenemos el término de antes como

$$z_1 = z_s - z_0 - z_2.$$

O sea: en vez de hacer 4 productos, hacemos unas sumas más y **solo 3 productos**. Notar que combinar se mantiene lineal.

Algoritmo de Karatsuba

La cantidad de operaciones que hace el algoritmo ahora satisface la relación⁴

$$T(n) = 3T(n/2) + \Theta(n).$$

Si analizamos el árbol de recursión podemos encontrar un nuevo candidato para la recursión (en este caso, es $\Theta(n^{\log_2 3})$). En vez de hacer eso, vamos a ver una herramienta que automatiza el razonamiento que estábamos haciendo.

⁴Alguien podría criticar que falta un +1 en la recursión porque los números son un poco más grandes que $n/2$, pero podemos absorberlo en el $\Theta(n)$ del final argumentando un poco.

El Teorema Maestro

Consideremos

$$T(n) = aT(n/b) + f(n),$$

donde $a \geq 1$, $b > 1$, $T(1) = \Theta(1)$ y f es eventualmente no negativa. Sea $q = \log_b a$. El teorema maestro dice que, dado $\varepsilon > 0$,

Caso	Hipótesis	Resultado
1	Si $f(n) = \mathcal{O}(n^{q-\varepsilon})$	$T(n) = \Theta(n^q)$
2	Si $f(n) = \Theta(n^q \log^r n)$ con $r \geq 0$,	$T(n) = \Theta(n^q \log^{r+1} n)$
3	Si $f(n) = \Omega(n^{q+\varepsilon})$ y $af(n/b) \leq cf(n)$ para algún $0 < c < 1$, eventualmente	$T(n) = \Theta(f(n))$

El Teorema Maestro

Caso	Hipótesis	Resultado
1	Si $f(n) = \mathcal{O}(n^{q-\varepsilon})$, luego	$T(n) = \Theta(n^q)$
2	Si $f(n) = \Theta(n^q \log^r n)$ con $r \geq 0$,	$T(n) = \Theta(n^q \log^{r+1} n)$
3	$f(n) = \Omega(n^{q+\varepsilon})$ y $af(n/b) \leq cf(n)$ para algún $0 < c < 1$, eventualmente	$T(n) = \Theta(f(n))$

Como vamos a ver más adelante, el caso 1 se corresponde con la situación en la que la mayoría del trabajo se hace en las recursiones. El caso 3 se corresponde con la situación en que la mayoría del trabajo se hace en el primer piso. Finalmente, el caso 2 se cumple cuando ambos costos son similares.

El Teorema Maestro

Supongamos que n es potencia de b (y entonces siempre se parte bien⁵). Tras j niveles,

$$T(n) = a^j T(n/b^j) + \sum_{i=0}^{j-1} a^i f(n/b^i).$$

Sea $L = \log_b n$. Al llegar a los casos base:

$$T(n) = \underbrace{a^L T(1)}_{\text{hojas}} + \underbrace{\sum_{i=0}^{L-1} a^i f(n/b^i)}_{\text{niveles internos}}.$$

Como

$$a^L = a^{\log_b n} = n^{\log_b a} = n^q,$$

las hojas cuestan $\Theta(n^q)$, y el nivel i cuesta $a^i f(n/b^i)$.

⁵El caso más general es similar, y se puede reducir a este ya que las diferencias se las come la complejidad asintótica)

El Teorema Maestro

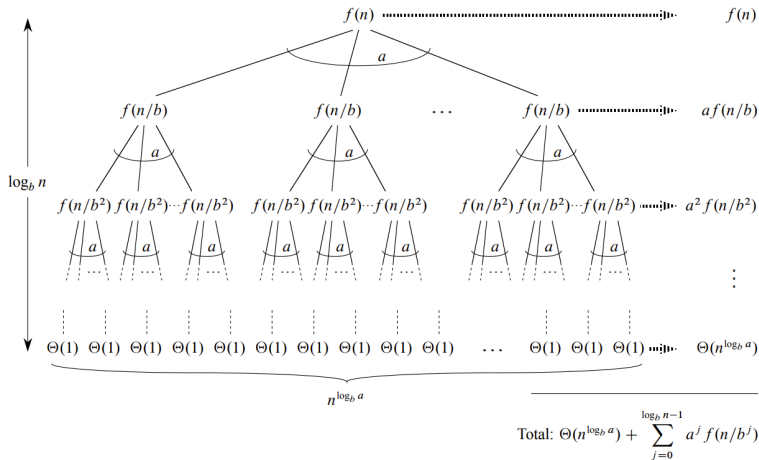


Figura: Diagrama del Teorema Maestro

El Teorema Maestro: Caso 1

Supongamos $f(n) = \mathcal{O}(n^{q-\varepsilon})$. Como $a = b^q$, el nivel i satisface para alguna constante C

$$\begin{aligned} a^i f(n/b^i) &\leq C a^i \left(\frac{n}{b^i}\right)^{q-\varepsilon} \\ &= C n^{q-\varepsilon} b^{\varepsilon i}. \end{aligned}$$

Con esto podemos acotar la suma geométrica como

$$\sum_{i=0}^{L-1} a^i f(n/b^i) \leq C n^{q-\varepsilon} \sum_{i=0}^{L-1} (b^\varepsilon)^i = \mathcal{O}(n^{q-\varepsilon} b^{\varepsilon L}) = \mathcal{O}(n^q).$$

Por lo tanto, en la expresión del costo el término de las hojas es dominante, y tenemos que:

$$T(n) = \Theta(n^q).$$

El Teorema Maestro: Caso 2

Supongamos $f(n) = \Theta(n^q \log^r n)$, con $r \geq 0$. En el nivel i :

$$\begin{aligned} a^i f(n/b^i) &= \Theta(a^i (n/b^i)^q \log^r(n/b^i)) \\ &= \Theta(n^q (L - i)^r), \end{aligned}$$

pues $a = b^q$ y $\log(n/b^i) = \Theta(L - i)$. Entonces

$$\begin{aligned} \sum_{i=0}^{L-1} a^i f(n/b^i) &= \Theta\left(n^q \sum_{s=1}^L s^r\right) \\ &= \Theta(n^q L^{r+1}). \end{aligned}$$

Como $L = \Theta(\log n)$,

$$T(n) = \Theta(n^q \log^{r+1} n).$$

El Teorema Maestro: Caso 3

Supongamos $f(n) = \Omega(n^{q+\varepsilon})$ y, para algún $0 < c < 1$ y todo n suficientemente grande,

$$af(n/b) \leq cf(n).$$

Iterando esta desigualdad, también sabemos que para n suficientemente grande vale que

$$a^i f(n/b^i) \leq c^i f(n).$$

Por lo tanto, aplicando esta desigualdad hasta el último piso donde se pueda (ya que solo vale a partir de un cierto n), podemos acotar

$$\sum_{i=0}^{L_0} a^i f(n/b^i) \leq f(n) \sum_{i \geq 0} c^i = \Theta(f(n)).$$

Los subárboles restantes aportan $\mathcal{O}(n^q)$, que es $\mathcal{O}(f(n))$. Concluimos entonces que

$$T(n) = \Theta(f(n)).$$

Los tres casos corresponden a tres formas del árbol

Caso 1

El costo aumenta hacia abajo.

Dominan las hojas

Caso 2

Los niveles son comparables.

Se acumulan niveles

Caso 3

El costo decrece hacia abajo.

Domina la raíz

La prueba es simplemente el razonamiento que veníamos haciendo, pero hecho de forma más general.

El algoritmo de Karatsuba

Apliquemos el teorema a la recursión de Karatsuba. La relación era

$$T(n) = 3T(n/2) + \Theta(n)$$

Entonces, tenemos

$$a = 3, \quad b = 2, \quad q = \log_2 3 \approx 1.585, \quad f(n) = \Theta(n).$$

Como $1 < \log_2 3$, existe un $\varepsilon > 0$ tal que:

$$n = \mathcal{O}\left(n^{\log_2 3 - \varepsilon}\right) \quad \text{con} \quad \varepsilon = 0.2 > 0.$$

Por el caso 1,

$$T(n) = \Theta(n^{\log_2 3}) \approx \mathcal{O}(n^{1.585}).$$

Mergesort de nuevo

Para practicar, apliquemos el Teorema Maestro a la recurrencia de Merge Sort

$$T(n) = 2T(n/2) + \Theta(n)$$

En este caso, tenemos

$$a = 2, \quad b = 2, \quad q = \log_2 2 = 1, \quad f(n) = \Theta(n)$$

Entonces caemos en el caso 2, con $r = 0$, y vale que

$$T(n) = \Theta(n \log n)$$

El algoritmo de Karatsuba

Unos comentarios interesantes:

- En Karatsuba mejoramos el algoritmo de producto partiendo los números de n bits en 2 de $n/2$. Mejorará la cosa si partimos en 3? Y en 4?

⁶Ver *Nature of Computation* de Moore para más :D

El algoritmo de Karatsuba

Unos comentarios interesantes:

- En Karatsuba mejoramos el algoritmo de producto partiendo los números de n bits en 2 de $n/2$. Mejorará la cosa si partimos en 3? Y en 4?
- Se puede! Y mejora. Es más, mientras más partes usamos, mejor. En general, podemos hacer un algoritmo $O(n^{1+\varepsilon})$ para cualquier $\varepsilon > 0$. Este algoritmo (o bien familia de algoritmos) se conoce como **Algoritmo de Toom-Cook**.

⁶Ver *Nature of Computation* de Moore para más :D

El algoritmo de Karatsuba

Unos comentarios interesantes:

- En Karatsuba mejoramos el algoritmo de producto partiendo los números de n bits en 2 de $n/2$. Mejorará la cosa si partimos en 3? Y en 4?
- Se puede! Y mejora. Es más, mientras más partes usamos, mejor. En general, podemos hacer un algoritmo $O(n^{1+\varepsilon})$ para cualquier $\varepsilon > 0$. Este algoritmo (o bien familia de algoritmos) se conoce como **Algoritmo de Toom-Cook**.
- Pero se puede hacer mejor: podemos elegir la cantidad de particiones de forma dinámica. Haciendo esto, se puede obtener una complejidad $O(n^{2\sqrt{2\log n}} \log n)$ (este algoritmo es de Knuth).

⁶Ver *Nature of Computation* de Moore para más :D

El algoritmo de Karatsuba

Unos comentarios interesantes:

- En Karatsuba mejoramos el algoritmo de producto partiendo los números de n bits en 2 de $n/2$. Mejorará la cosa si partimos en 3? Y en 4?
- Se puede! Y mejora. Es más, mientras más partes usamos, mejor. En general, podemos hacer un algoritmo $O(n^{1+\varepsilon})$ para cualquier $\varepsilon > 0$. Este algoritmo (o bien familia de algoritmos) se conoce como **Algoritmo de Toom-Cook**.
- Pero se puede hacer mejor: podemos elegir la cantidad de particiones de forma dinámica. Haciendo esto, se puede obtener una complejidad $O(n^{2\sqrt{2\log n}} \log n)$ (este algoritmo es de Knuth).
- Y se puede mejorar más: reduciendo el problema a multiplicación de matrices, la complejidad se puede bajar a $O(n \log n \log \log n)$ (Strassen).

⁶Ver *Nature of Computation* de Moore para más :D

El algoritmo de Karatsuba

Unos comentarios interesantes:

- En Karatsuba mejoramos el algoritmo de producto partiendo los números de n bits en 2 de $n/2$. Mejorará la cosa si partimos en 3? Y en 4?
- Se puede! Y mejora. Es más, mientras más partes usamos, mejor. En general, podemos hacer un algoritmo $O(n^{1+\varepsilon})$ para cualquier $\varepsilon > 0$. Este algoritmo (o bien familia de algoritmos) se conoce como **Algoritmo de Toom-Cook**.
- Pero se puede hacer mejor: podemos elegir la cantidad de particiones de forma dinámica. Haciendo esto, se puede obtener una complejidad $O(n2^{\sqrt{2\log n}} \log n)$ (este algoritmo es de Knuth).
- Y se puede mejorar más: reduciendo el problema a multiplicación de matrices, la complejidad se puede bajar a $O(n \log n \log \log n)$ (Strassen).
- Lo óptimo se cree que es $O(n \log n)$ (conjetura de Kolmogorov, entre otros), y fue alcanzado en 2019. La constante escondida es tan grande que para que el algoritmo sea práctico la entrada tiene que tener tamaño por lo menos $2^{10^{38}}$.⁶

⁶Ver *Nature of Computation* de Moore para más :D

Pausa

Pausa

El algoritmo de Strassen

Problema: multiplicar matrices

Dadas dos matrices A y B , queremos calcular $C = AB$.

Usando el algoritmo del CBC... Qué complejidad alcanzamos?

El algoritmo de Strassen

Problema: multiplicar matrices

Dadas dos matrices A y B , queremos calcular $C = AB$.

Usando el algoritmo del CBC... Qué complejidad alcanzamos?
Es $O(n^3)$. Para mejorarlo, vamos a usar de vuelta D&C.

Algoritmo de Strassen

Dividimos las matrices en bloques,

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}.$$

Y podemos expresar cada bloque de C en término de productos más chicos. Por ejemplo

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}, \quad C_{12} = A_{11}B_{12} + A_{12}B_{22},$$

y análogamente para C_{21} y C_{22} .

En este algoritmo, qué complejidad tiene combinar?

Algoritmo de Strassen

Dividimos las matrices en bloques,

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}.$$

Y podemos expresar cada bloque de C en término de productos más chicos. Por ejemplo

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}, \quad C_{12} = A_{11}B_{12} + A_{12}B_{22},$$

y análogamente para C_{21} y C_{22} .

En este algoritmo, qué complejidad tiene combinar? $O(n^2)$, o sea, lineal con respecto a la entrada.

La recurrencia de la cantidad de operaciones es

$$T(m) = 8T(m/4) + \Theta(m)$$

con $m = n^2$. Usando Maestro, sale que la complejidad es $\Theta(m^{3/2}) = \Theta((n^2)^{3/2}) = \Theta(n^3)$. O sea, no mejoramos nada.

El algoritmo de Strassen

Con mucha inspiración, es posible darse cuenta, como hizo Strassen, de que con 7 multiplicaciones podemos recuperar todos los bloques de C . Estos son los elegidos:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22}),$$

$$M_2 = (A_{21} + A_{22})B_{11},$$

$$M_3 = A_{11}(B_{12} - B_{22}),$$

$$M_4 = A_{22}(B_{21} - B_{11}),$$

$$M_5 = (A_{11} + A_{12})B_{22},$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12}),$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22}).$$

Por ejemplo, $C_{11} = M_1 + M_7 + M_4 - M_5$ (cuidado, lo revisé a mano).

Esto reduce el problema de multiplicar dos matrices de $n \times n$ a multiplicar 7 de $\sim n/2 \times n/2$ y hacer $O(1)$ sumas que se pueden hacer en $O(n^2)$. Luego, la recurrencia queda como:

$$T(n^2) = 7T(n^2/4) + \Theta(n^2)$$

Algoritmo de Strassen

Llamando $m = n^2$,

$$T(m) = 7T(m/4) + \Theta(m)$$

Las variables del Teorema Maestro quedan como

$$a = 7, \quad b = 4, \quad q = \log_4 7 \sim 1.41, \quad f(m) = \Theta(m)$$

Como $f(m) = m = O(m^{q-0.1})$ estamos en el primer caso, y la complejidad es $T(m) = \Theta(m^q) = \mathcal{O}((n^2)^{1.41}) = \mathcal{O}(n^{2.82})$. Mejoramos!

El algoritmo de Strassen

Algunos comentarios adicionales:

- Se podrá hacer mejor? Onda, partir en más subbloques y encontrar (mágicamente) nuevas reescrituras?

El algoritmo de Strassen

Algunos comentarios adicionales:

- Se podrá hacer mejor? Onda, partir en más subbloques y encontrar (mágicamente) nuevas reescrituras?
- Si y no. Ahora no es tan sencillo “descubrir” los subbloques, ni aprender como pegarlos. Strassen (con herramientas de Schönhage) demostró que, en el fondo, su algoritmo se corresponde con encontrar una **descomposición tensorial** del producto.

El algoritmo de Strassen

Algunos comentarios adicionales:

- Se podrá hacer mejor? Onda, partir en más subbloques y encontrar (mágicamente) nuevas reescrituras?
- Si y no. Ahora no es tan sencillo “descubrir” los subbloques, ni aprender como pegarlos. Strassen (con herramientas de Schönhage) demostró que, en el fondo, su algoritmo se corresponde con encontrar una **descomposición tensorial** del producto.
- El juego entonces es encontrar mejores descomposiciones tensoriales.

Autores	Año	Complejidad
Strassen	1969	$O(n^{2.8074})$
Coppersmith–Winograd	1990	$O(n^{2.375477+\varepsilon})$
Vassilevska Williams	2012	$O(n^{2.3729+\varepsilon})$
Le Gall	2014	$O(n^{2.372864+\varepsilon})$
Duan–Wu–Zhou	2023	$O(n^{2.371866+\varepsilon})$
Vassilevska Williams–Xu–Xu–Zhou	2024	$O(n^{2.371552+\varepsilon})$
Dupont et al. (<i>preprint</i>)	2026	$O(n^{2.371177+\varepsilon})$

Timba

Problema: Timbear

Tenemos predicciones para el precio de una acción para distintos días como $p_1 \dots p_n$.
Tenemos que elegir un día i para comprar, y uno j para vender. Naturalmente, queremos maximizar la ganancia.

Formalmente, buscamos $i < j$ tal que

$$p_j - p_i = \max_{1 \leq a < b \leq n} \{p_b - p_a\}$$

Cuál es el algoritmo obvio? Qué complejidad tiene?

Timba

Problema: Timbear

Tenemos predicciones para el precio de una acción para distintos días como $p_1 \dots p_n$. Tenemos que elegir un día i para comprar, y uno j para vender. Naturalmente, queremos maximizar la ganancia.

Formalmente, buscamos $i < j$ tal que

$$p_j - p_i = \max_{1 \leq a < b \leq n} \{p_b - p_a\}$$

Cuál es el algoritmo obvio? Qué complejidad tiene?

El problema se puede resolver fácilmente en $O(n^2)$. Intentemos hacerlo más rápido.

Timba

Vamos a usar una estrategia muy útil para resolver problemas: reducirlo a otro (que con un poco de suerte ya resolvió otra persona, o bien es más fácil). Definamos

$$d_k = p_{k+1} - p_k, \quad 1 \leq k < n.$$

d_k indica la ganancia que obtenemos si compramos al día k y vendemos al $k + 1$. Si compramos el día i y vendemos el día j , vale que

$$\begin{aligned} p_j - p_i &= (p_{i+1} - p_i) + (p_{i+2} - p_{i+1}) + \cdots + (p_j - p_{j-1}) \\ &= \sum_{k=i}^{j-1} d_k. \end{aligned}$$

O sea, los índices que maximizan $p_j - p_i$ son exactamente los que maximizan $\sum_{k=i}^{j-1} d_k$.

Máximo subarreglo

Más formalmente, tenemos la siguiente proposición

Proposición

$$\max_{1 \leq i < j \leq n} (p_j - p_i) = \max_{1 \leq \ell \leq r < n} \sum_{k=\ell}^r d_k.$$

En particular, si (i, j) es una solución al problema de la timba, $(i, j - 1)$ es una solución del problema de máximo subarreglo, y hay una traducción análoga en la otra dirección.

Demostración. Es inmediata de la igualdad con la suma telescópica que hicimos antes.

En base a este resultado, podemos olvidarnos de la timba y quedarnos con el problema del máximo subarreglo.

Máximo subarreglo

Intentemos proponer un enfoque con D&C para resolver el problema. Qué podríamos hacer?

Máximo subarreglo

Intentemos proponer un enfoque con D&C para resolver el problema. Qué podríamos hacer? Dividir el arreglo a la mitad, hacer recursión de cada lado, luego al combinar buscar el máximo subarreglo que pasa por el medio. Más formalmente, si buscamos el máximo subarreglo entre dos índices ℓ y r , hay tres opciones, con $m = (\ell + r)/2$:

Izquierda

Está completamente contenido en $[\ell, m]$.

Derecha

Está completamente contenido en $[m + 1, r]$.

Cruza

Cruza el centro: comienza en $i \leq m$ y termina en $j \geq m + 1$.

Intuitivamente

$$\text{MAXSUB}(\ell, r) = \max\{\text{MAXSUB}(\ell, m), \text{MAXSUB}(m + 1, r), \text{CRUZA}(\ell, m, r)\}.$$

Máximo subarreglo

Para la parte de combinar, alcanza con observar que hay que maximizar para la izquierda y luego para la derecha:

$$\begin{aligned}\text{CRUZA}(\ell, m, r) &= \max_{\ell \leq i \leq m < j \leq r} \sum_{k=i}^j d_k \\ &= \max_{\ell \leq i \leq m} \sum_{k=i}^m d_k + \max_{m < j \leq r} \sum_{k=m+1}^j d_k.\end{aligned}$$

En qué complejidad podemos calcular esto?

Máximo subarreglo

Para la parte de combinar, alcanza con observar que hay que maximizar para la izquierda y luego para la derecha:

$$\begin{aligned}\text{CRUZA}(\ell, m, r) &= \max_{\ell \leq i \leq m < j \leq r} \sum_{k=i}^j d_k \\ &= \max_{\ell \leq i \leq m} \sum_{k=i}^m d_k + \max_{m < j \leq r} \sum_{k=m+1}^j d_k.\end{aligned}$$

En qué complejidad podemos calcular esto? En $\Theta(r - \ell)$, es un recorrido manteniendo el mejor sufijo / prefijo.

Máximo subarreglo

Hacemos que los llamados recursivos devuelvan el valor máximo y también el intervalo que lo alcanza.

MAX-SUBARRAY(d, ℓ, r)

if $\ell = r$: **return** ($[\ell, \ell], d_\ell$)

$m \leftarrow \lfloor (\ell + r)/2 \rfloor$

$L \leftarrow \text{MAX-SUBARRAY}(d, \ell, m)$

$R \leftarrow \text{MAX-SUBARRAY}(d, m + 1, r)$

$C \leftarrow \text{MAX-CROSSING}(d, \ell, m, r)$

return el candidato de mayor suma entre L, R, C

Correctitud: por inducción, dado que solo hay 3 casos posibles (y tomamos el máximo entre los 3).

Máximo subarreglo

Complejidad La cantidad de operaciones que hace el algoritmo satisface la recursión

$$T(n) = 2T(n/2) + \Theta(n).$$

Podemos usar el Teorema Maestro (aunque esta recursión ya apareció antes)

$$a = 2, \quad b = 2, \quad q = 1, f(n) = \Theta(n)$$

y por lo tanto caemos en el caso 2 con $r = 0$: la complejidad final queda

$$T(n) = \Theta(n \log n).$$

Máximo subarreglo

Se puede hacer mejor? Hay alguna parte en donde estemos siendo redundantes?

Máximo subarreglo

Se puede hacer mejor? Hay alguna parte en donde estemos siendo redundantes?

En cada llamado recursivo se recorre todo el arreglo, y después cuando buscamos el máximo subarreglo que cruza volvemos a recorrer el arreglo. No podrían los llamados recursivos ayudarnos?

Máximo subarreglo

Se puede hacer mejor? Hay alguna parte en donde estemos siendo redundantes?

En cada llamado recursivo se recorre todo el arreglo, y después cuando buscamos el máximo subarreglo que cruza volvemos a recorrer el arreglo. No podrían los llamados recursivos ayudarnos?

Idea: que cada llamado devuelva los índices de su mejor subarreglo, su valor, pero también los máximos subarreglos que empiezan **desde cada borde**.

Máximo subarreglo

Para un segmento no vacío $X = d_\ell, \dots, d_r$ definimos

$$\text{tot}(X) = \sum_{k=\ell}^r d_k,$$

$$\text{pre}(X) = \max_{\ell \leq q \leq r} \sum_{k=\ell}^q d_k,$$

$$\text{suf}(X) = \max_{\ell \leq q \leq r} \sum_{k=q}^r d_k,$$

$$\text{best}(X) = \max_{\ell \leq u \leq v \leq r} \sum_{k=u}^v d_k.$$

Para un singleton $X = (x)$,

$$\text{tot}(X) = \text{pre}(X) = \text{suf}(X) = \text{best}(X) = x.$$

Es decir, para el caso base podemos calcular todo fácilmente.

Máximo subarreglo

Sea $X = LR$, con L y R consecutivos. Entonces

$$\text{tot}(X) = \text{tot}(L) + \text{tot}(R),$$

Máximo subarreglo

Sea $X = LR$, con L y R consecutivos. Entonces

$$\text{tot}(X) = \text{tot}(L) + \text{tot}(R),$$

$$\text{pre}(X) = \max\{\text{pre}(L), \text{tot}(L) + \text{pre}(R)\},$$

Máximo subarreglo

Sea $X = LR$, con L y R consecutivos. Entonces

$$\text{tot}(X) = \text{tot}(L) + \text{tot}(R),$$

$$\text{pre}(X) = \max\{\text{pre}(L), \text{tot}(L) + \text{pre}(R)\},$$

$$\text{suf}(X) = \max\{\text{suf}(R), \text{suf}(L) + \text{tot}(R)\}$$

Máximo subarreglo

Sea $X = LR$, con L y R consecutivos. Entonces

$$\text{tot}(X) = \text{tot}(L) + \text{tot}(R),$$

$$\text{pre}(X) = \max\{\text{pre}(L), \text{tot}(L) + \text{pre}(R)\},$$

$$\text{suf}(X) = \max\{\text{suf}(R), \text{suf}(L) + \text{tot}(R)\}$$

$$\text{best}(X) = \max\{\text{best}(L), \text{best}(R), \text{suf}(L) + \text{pre}(R)\}$$

Hay que justificar bien estas igualdades, capturan la correctitud del algoritmo recursivo.

Máximo subarreglo

Con estos valores nos ahorramos gastar $\Theta(n)$ en la parte de combinar, y en cambio sale en $O(1)$. Luego, la recursión de la complejidad queda como

$$T(n) = 2T(n/2) + \Theta(1).$$

Por el caso 1 del Teorema Maestro,

$$T(n) = \Theta(n).$$

lo cual es óptimo! **Tarea:** escribir bien el algoritmo para que devuelva también el intervalo que maximiza la suma (y no solo el valor máximo)

Búsqueda binaria

La búsqueda binaria es un caso de D&C

BINARY-SEARCH(A, x, ℓ, r)

Precondición: $A[\ell : r]$ está ordenado

if $\ell > r$: **return** NO-ESTÁ

$m \leftarrow \lfloor (\ell + r)/2 \rfloor$

if $A[m] = x$: **return** m

if $x < A[m]$:

return BINARY-SEARCH($A, x, \ell, m - 1$)

si no:

return BINARY-SEARCH($A, x, m + 1, r$)

La complejidad respeta una recursión de la forma

$$T(n) \leq T(n/2) + \Theta(1)$$

y usando el Teorema Maestro se puede ver que esto es $O(\log n)$.

Más allá de búsqueda binaria

Pero la idea de búsqueda binaria va más allá de un arreglo. Basta tener:

- un dominio ordenado de candidatos T ;
- un predicado $P(t)$ que queremos evaluar; y
- un único umbral t^* donde el predicado cambia

$$P(t) = \begin{cases} \text{falso}, & t < t^*, \\ \text{verdadero}, & t \geq t^*. \end{cases}$$

En general vamos a poder hacer búsqueda binaria para encontrar el menor t para el que la propiedad se cumple en $O(\log |T|)$ (asumiendo que evaluar P toma $O(1)$).

Encuentro

Problema

Tenemos n amigos distribuidos en distintas posiciones $p_1 \dots p_n$ en una recta. Cada amigo tiene una cierta velocidad máxima a la que se puede mover v_i (en ambos sentidos). La pregunta es: cuál es el menor tiempo t tal que todos los amigos se pueden encontrar?

Encuentro

Problema

Tenemos n amigos distribuidos en distintas posiciones $p_1 \dots p_n$ en una recta. Cada amigo tiene una cierta velocidad máxima a la que se puede mover v_i (en ambos sentidos). La pregunta es: cuál es el menor tiempo t tal que todos los amigos se pueden encontrar?

Asumamos que las posiciones están en metros, que el tiempo se mide en segundos, que la velocidad está expresada en metros por segundos y que hay un máximo tiempo (en segundos) T . Aparte, supongamos que nos piden el menor segundo a partir del cuál se pueden encontrar.

Hay algún algoritmo fácil que resuelva el problema?

Encuentro

Intentemos repensar lo que nos piden. Que se puedan encontrar en el momento t es equivalente a que exista un punto p tal que todos puedan acercarse en t o menos segundos. Dado un amigo i , podemos calcular qué posiciones puede alcanzar en t o menos segundos?

Encuentro

Intentemos repensar lo que nos piden. Que se puedan encontrar en el momento t es equivalente a que exista un punto p tal que todos puedan acercarse en t o menos segundos. Dado un amigo i , podemos calcular qué posiciones puede alcanzar en t o menos segundos? Es exactamente el intervalo

$$I_{i,t} = [p_i - v_i t, p_i + v_i t]$$

Luego... Cuándo vale que existe un punto en el que todos pueden encontrarse en t o menos segundos?

Encuentro

Intentemos repensar lo que nos piden. Que se puedan encontrar en el momento t es equivalente a que exista un punto p tal que todos puedan acercarse en t o menos segundos. Dado un amigo i , podemos calcular qué posiciones puede alcanzar en t o menos segundos? Es exactamente el intervalo

$$I_{i,t} = [p_i - v_i t, p_i + v_i t]$$

Luego... Cuándo vale que existe un punto en el que todos pueden encontrarse en t o menos segundos? Tiene que haber un punto en común en esta intersección. Es decir,

$$\bigcap_{i=1}^n I_{i,t} \neq \emptyset$$

Encuentro

Ahora si podemos proponer un primer algoritmo.

- 1 Iterar por cada $0 \leq t \leq T$.
- 2 Por cada t ver si $\bigcap_{i=1}^n I_{i,t} \neq \emptyset$ y quedarnos con el más chico (si no hay ninguno, los amigos no pueden encontrarse).

El algoritmo es correcto: hace búsqueda sobre cada posible valor de t , y para cada uno revisa exactamente si los amigos pueden encontrarse. **Complejidad:**

Encuentro

Ahora si podemos proponer un primer algoritmo.

- 1 Iterar por cada $0 \leq t \leq T$.
- 2 Por cada t ver si $\bigcap_{i=1}^n I_{i,t} \neq \emptyset$ y quedarnos con el más chico (si no hay ninguno, los amigos no pueden encontrarse).

El algoritmo es correcto: hace búsqueda sobre cada posible valor de t , y para cada uno revisa exactamente si los amigos pueden encontrarse. **Complejidad:** $O(nT)$ si implementamos prolijamente la intersección de intervalos.

Se puede mejorar?

Encuentro

Notemos que hay un primer punto t a partir del cual los amigos siempre pueden encontrarse. Esto es porque $I_{i,t} \subseteq I_{i,t+1}$ (o, en criollo, si te doy más tiempo siempre puedes cubrir más terreno). Luego, podemos hacer **búsqueda binaria** sobre $[0 \dots T]$.

Agregando esta mejora, la complejidad baja a $O(n \log T)$, lo cual alcanza para pasar este juez -> <https://codeforces.com/problemset/problem/782/B>

Conclusión

- La técnica de D&C permite resolver algunos problemas. La idea central es subdividir el problema en subproblemas, y luego combinar las soluciones obtenidas recursivamente.

⁷Si los subproblemas no tienen el mismo tamaño se puede usar el Teorema de Akra-Bazzi, que generaliza el Maestro

Conclusión

- La técnica de D&C permite resolver algunos problemas. La idea central es subdividir el problema en subproblemas, y luego combinar las soluciones obtenidas recursivamente.
- Generalmente las demostraciones de correctitud se hacen por inducción, y los cálculos de complejidad usando el Teorema Maestro⁷.

⁷Si los subproblemas no tienen el mismo tamaño se puede usar el Teorema de Akra-Bazzi, que generaliza el Maestro

Conclusión

- La técnica de D&C permite resolver algunos problemas. La idea central es subdividir el problema en subproblemas, y luego combinar las soluciones obtenidas recursivamente.
- Generalmente las demostraciones de correctitud se hacen por inducción, y los cálculos de complejidad usando el Teorema Maestro⁷.
- La tensión de la complejidad está entre el costo de los llamados recursivos y el costo de la combinación. Identificar cuál es el cuello de botella es relevante para entender qué hay que optimizar (ver el caso de máximo subarreglo).

⁷Si los subproblemas no tienen el mismo tamaño se puede usar el Teorema de Akra-Bazzi, que generaliza el Maestro

Conclusión

- La técnica de D&C permite resolver algunos problemas. La idea central es subdividir el problema en subproblemas, y luego combinar las soluciones obtenidas recursivamente.
- Generalmente las demostraciones de correctitud se hacen por inducción, y los cálculos de complejidad usando el Teorema Maestro⁷.
- La tensión de la complejidad está entre el costo de los llamados recursivos y el costo de la combinación. Identificar cuál es el cuello de botella es relevante para entender qué hay que optimizar (ver el caso de máximo subarreglo).
- La búsqueda binaria es un tipo de algoritmo de D&C, y tiene muchas más aplicaciones que hacer búsquedas en bases de datos!

⁷Si los subproblemas no tienen el mismo tamaño se puede usar el Teorema de Akra-Bazzi, que generaliza el Maestro